

PROJECT REPORT

AK CLICKS: Photography Booking and The Story Shop Platform

A combined Flask web application for event photography bookings, book commerce, customer accounts and studio operations

Prepared for	Technology	Project type
AK CLICKS	Python Flask, SQLite, HTML/CSS/JavaScript	Full-stack portfolio, booking and commerce website

July 2026

Executive Summary

AK CLICKS is a full-stack web platform that combines a photography portfolio and event booking system with The Story Shop, an online bookstore. Customers can explore photography services, submit wedding, birthday, event, portrait or brand-content enquiries, purchase books, create accounts and download test payment receipts. Studio administrators use one dashboard to manage bookings, orders, inventory, payment submissions and queued notifications.

Project Title

AK CLICKS - Integrated Photography Booking and Story Shop Management System

Objectives

- Present photography work in a professional responsive portfolio.
- Capture online enquiries for weddings, birthdays and other events.
- Sell Story Shop books through a stock-aware shopping bag and checkout flow.
- Provide a shared admin workspace for bookings, orders, stock, payment state and notifications.
- Create a foundation for live payments, email and WhatsApp delivery.

Problem Statement

Photography businesses often handle enquiries through calls and messages, while independent shops track orders and inventory separately. This creates missed dates, manual stock errors and inconsistent customer communication. The project centralises these workflows in a single browser-based application.

Scope and User Roles

Role	Key capabilities
Visitor	View photography portfolio, browse books, check a date and submit booking/order details.
Customer	Register, sign in and view their Story Shop order history.
Administrator	Approve or decline bookings; process orders; update book stock; review payment state and notification queue.

Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Responsive interface, bag management, form controls and payment-method switching.
Backend	Python 3 with Flask	Routes, validation, sessions, payment pages, PDF receipt generation and admin access.
Database	SQLite	Persistent bookings, orders, customers, products, stock and notifications.
Security utilities	Werkzeug password hashing	Stores customer passwords as hashes rather than plain text.
QR and receipts	qrcode, ReportLab	Local SVG UPI QR generation and downloadable PDF test receipts.

System Architecture

The browser renders Jinja HTML templates served by Flask. Forms submit to Flask routes, which validate data and write to SQLite. The same data is read by the customer account and administrator dashboard. JavaScript keeps the temporary shopping bag in browser local storage until checkout. The architecture is intentionally simple for local deployment and can later be moved to PostgreSQL and a managed hosting service.

Component	Responsibility	Primary data
Public site	Portfolio, booking form, availability check	Bookings
Story Shop	Catalog, bag, checkout	Products, orders, stock
Payment module	Test payment method, QR, receipt	Payment method/status
Account module	Registration, login, order history	Customers, orders
Admin dashboard	Operations control	All operational records

Functional Modules

Photography Booking Module

- Services include wedding, birthday, event, portrait and brand photography.
- The selected event date can be checked through the availability endpoint. A date with a non-declined booking is reported as already having an enquiry.
- A valid enquiry stores customer contact details, service type, event date, location and notes, then opens the booking-advance payment page.

Story Shop Module

- The integrated catalog contains six books and uses the original Story Shop artwork.
- The bag calculates the total in the browser; the server recalculates the final total and checks stock before creating an order.
- Stock is decremented only after a valid order is created, preventing orders for unavailable quantities.

Customer Account Module

- New customers can register with name, email and a minimum six-character password.
- Passwords are protected with Werkzeug password hashes.
- Signed-in customers can review orders associated with their account.

Payment and Receipt Module

The payment screens provide UPI/QR, net-banking and card user interfaces. They are intentionally labelled TEST MODE: no financial data is transferred and no real money is collected. After a customer

selects a method, the system records a test payment submission and creates a PDF receipt that explicitly states it is not proof of a financial transaction.

Database Design

Table	Purpose	Important fields
bookings	Photography enquiries	name, email, phone, service, booking_date, status, payment_status
orders	Story Shop purchases	customer_name, address, items, total, status, customer_id
customers	Customer identities	name, email (unique), password_hash
products	Book catalog and inventory	id, name, price, stock, image, description
notifications	Queued communications	recipient, channel, subject, body, status

Routes and API Reference

Method	Route	Purpose
GET	/	Photography portfolio and booking form
POST	/book	Creates booking enquiry and opens booking payment
GET	/availability?date=YYYY-MM-DD	JSON date-availability response
GET	/shop	Story Shop catalog and bag
POST	/order	Validates stock, creates order and reserves inventory
GET/POST	/account	Customer registration and login
GET	/payment/<kind>/<id>	Payment-selection page
GET	/payment/<kind>/<id>/qr	SVG test-mode UPI QR code
POST	/payment/<kind>/<id>/confirm	Stores test payment submission
GET	/receipt/<kind>/<id>.pdf	Downloads a test payment receipt
GET	/admin	Protected studio dashboard

Workflow: Booking to Confirmation

1. Customer completes the booking form.
2. Flask validates required fields and writes a bookings record.
3. The system queues a confirmation notification record.
4. Customer selects a test payment method for the configured booking advance.
5. The status and method are saved; a receipt can be downloaded.
6. Administrator reviews the enquiry and approves, declines or leaves it pending.

Workflow: Order to Fulfilment

7. Customer adds books to the browser bag.
8. Checkout submits contact and delivery details.
9. Server checks each product against current stock, recalculates the total and decrements inventory.
10. Order and queued confirmation record are created.
11. Customer performs a test payment submission and downloads a receipt.
12. Administrator changes order status to New, Processing or Fulfilled.

Notification Design

Email and WhatsApp notifications are implemented as a local queue. Booking, order and test payment actions insert notification records visible in the admin dashboard. This approach proves the business workflow without sending external messages or requiring credentials. A production integration should use SMTP or a transactional email provider plus WhatsApp Business Cloud API or Twilio, using credentials stored only as environment variables.

Security and Validation

- Administrator dashboard is protected by a session login; production credentials must replace the current development defaults.
- Customer passwords are hashed with Werkzeug.
- Checkout recalculates price and stock on the server rather than trusting the client total.
- Payment forms are explicitly test mode and must never collect actual card details until a PCI-compliant gateway is integrated.
- Production deployment must use HTTPS, a strong SECRET_KEY, CSRF protection, secure cookies and a production database.

Testing Performed

Area	Verification
Application syntax	Python compilation completed successfully.
Customer account	Registration and account-page access tested using an isolated in-memory database.
Shop and stock-aware order	Catalog response, valid order creation and inventory validation tested.
Availability API	JSON response checked for an unused date.
Payment and QR	Payment page, confirmation route and SVG QR endpoint checked.
Receipt	PDF receipt response generated and confirmed as application/pdf.

Limitations and Production Roadmap

Current state	Production next step
Test-mode payment UI and QR	Integrate Razorpay/Stripe and use verified webhooks before marking payments as paid.
Queued notifications	Configure SMTP/SendGrid and WhatsApp Business/Twilio credentials.
SQLite local database	Use PostgreSQL with backups and migration tooling.
Simple date availability	Add time slots, staff/resource capacity, calendar UI and calendar sync.
Local server	Deploy behind HTTPS on a managed host with environment secrets.
Basic customer area	Add password reset, verified email, booking history and profile editing.

Conclusion

The AK CLICKS platform demonstrates a practical integrated web application for a photography business and an independent book store. It turns portfolio visits into trackable booking enquiries, turns book selections into stock-aware orders, and gives studio staff one operational view. The project is suitable

for demonstration and local use today, with a clear path to live payments, external communications and production deployment.

Appendix A: Diagrams, Test Cases and Core Code

The following diagrams are provided in Mermaid notation. They can be rendered in GitHub, VS Code Markdown Preview, or Mermaid Live Editor.

Use Case Diagram

```
flowchart LR
    Visitor --> ViewPortfolio
    Visitor --> SubmitBooking
    Visitor --> BrowseStoryShop
    Customer --> RegisterLogin
    Customer --> PlaceOrder
    Administrator --> ManageBookings
    Administrator --> UpdateStock
    Administrator --> ReviewNotifications
```

Sequence Diagram - Book Order

```
sequenceDiagram
    Customer->>Browser: Add books to bag
    Browser->>Flask: POST /order
    Flask->>SQLite: Check stock and calculate total
    SQLite-->>Flask: Availability result
    Flask->>SQLite: Create order and reduce stock
    Flask-->>Browser: Redirect to payment
    Browser->>Flask: Confirm test payment
    Flask->>SQLite: Save payment status and queue notice
```

Activity Diagram / Flowchart

```
flowchart TD
    Start --> ChooseJourney
    ChooseJourney -->|Photography| BookingForm
    ChooseJourney -->|Story Shop| ShoppingBag
    BookingForm --> SaveBooking
    ShoppingBag --> CheckStock
    CheckStock --> SaveOrder
    SaveBooking --> Payment
    SaveOrder --> Payment
    Payment --> NotificationQueue
    NotificationQueue --> AdminProcessing
```

System Architecture Diagram

```
flowchart TB
    BrowserUI["Browser UI: HTML CSS JavaScript"] --> Flask["Flask Application"]
    Flask --> Templates["Jinja Templates"]
    Flask --> SQLite["SQLite Database"]
    SQLite --> Bookings
    SQLite --> Orders
    SQLite --> ProductsStock
    SQLite --> Customers
    SQLite --> Notifications
    Flask --> QR["QR Generator"]
    Flask --> Receipt["ReportLab PDF Receipts"]
```

Workflow Chart

```
flowchart LR
    Discover --> BookingOrOrder
```

```
BookingOrder --> Validation
Validation --> SQLiteStorage
SQLiteStorage --> TestPayment
TestPayment --> NotificationQueue
NotificationQueue --> AdminDashboard
AdminDashboard --> ApprovalOrFulfilment
```

Real-Time Performance Considerations

- Portfolio and shop pages are served locally through Flask templates and static assets.
- The availability endpoint runs a lightweight SQLite count query.
- Order checkout validates stock and recalculates totals on the server.
- QR codes and PDF receipts are generated on demand.
- Production monitoring should measure page load, endpoint latency, database errors and notification delivery.

Test Case Table

ID	Test case	Expected result
TC-01	Submit valid booking	Booking record is saved and payment page opens.
TC-02	Check unused date	Availability API returns available true.
TC-03	Register customer	Password hash is stored and session starts.
TC-04	Order in-stock books	Order saves and stock decreases.
TC-05	Order unavailable stock	Order is rejected without stock change.
TC-06	Confirm payment method	Test payment state and method are stored.
TC-07	Download receipt	PDF receipt response is returned.
TC-08	Open admin without login	User is redirected to login.
TC-09	Update stock	Admin inventory amount is stored.
TC-10	Create order/booking	Notification queue receives a record.

Core Code Extracts

Booking creation route

```
@app.route("/book", methods=["POST"])
def book():
    fields = {key: request.form.get(key, "").strip() for key in fields}
    with db_connection() as conn:
        cursor = conn.execute("INSERT INTO bookings (...) VALUES (...)", fields)
        booking_id = cursor.lastrowid
    return redirect(url_for("payment", kind="booking", record_id=booking_id))
```


Stock validation

```
requested = Counter(item.strip() for item in customer["items"].split(",") if item.strip())
all_products = {row["name"]: row for row in conn.execute("SELECT * FROM products")}
for name, quantity in requested.items():
    conn.execute("UPDATE products SET stock=stock-? WHERE id=?", (quantity, all_products[name]["id"]))
```

Availability API

```
@app.route("/availability")
def availability():
    date = request.args.get("date", "")
    count = conn.execute("SELECT COUNT(*) FROM bookings WHERE booking_date=?", (date,)).fetchone()[0]
    return jsonify({"date": date, "available": count == 0})
```